PUNTO 4 - TALLER 3 - INTELIGENCIA ARTIFICIAL

REFLEXION SOBRE EL TRABAJO REALIZADO

Integrantes:

- Daniela Munar Gonzáles - 202311392
- Juan Sebastián Parra Casas – 202411217
- Camilo Puerto Peña - 202211362
- Tomas Loaiza Rodríguez – 202512388

REFLEXION PUNTO 1

- **Get_all_models**

Para la implementación de esta función me enfoqué en la generación de combinaciones binarias, por lo que al obtener la lista de átomos y calcular la cantidad total de modelos posibles como **2^n** siendo **n el número de átomos**, hice una iteración para que al pasar por cada número sacara su representación binaria con tal de asignar valores de verdad a cada átomo dentro de un diccionario.

La dificultad principal que tuve al momento de la implementación, fue estructurar de manera correcta la generación de lo modelos, pues al principio se sobreescribían los valores de los modelos debido a que reutilizaba la misma variable **model** y no se creaba un nuevo diccionario en cada iteración, sino que lo reiniciaba de nuevo. Esto ocasionaba que los modelos no se almacemaran, por lo que en la revisión del código con ayuda de Inteligencia Artificial (de la cual se hablará mas adelante), me resaltó que este era el principal error el cual impedia que cada modelo fuera independiente.

Reflexion de uso de IA

Prompt Utilizado con Chat GPT:

Porfavor colabórame a corregir el siguiente codigo mostrándome los errores que puede haber y además me indicas cual es la correción que realizaste en forma de comentarios. También porfavor optimizala en dado caso que se pueda.

La IA se utilizó con tal de corregir y mejorar el código incial el cual yo habia desarrollado con base en las intrucciones, la pista dada y los notebooks de información proporcionados. Principalmente fueron errores y ajustes estructurales sin cambiar el enfoque que tenía, no obstante, y recordando lo mencionado anteriormente, uno de los errores principales era que la IA me recalcó que estaba usando una misma variable para guardar los modelos, lo que los sobreescribía en cada iteración impidiendo que hubieran mas modelos independientes.

Pienso que los beneficios de la IA en este contexto es precisamente que es de ayuda para identificar errores cruciales o mejorar el codigo, pero las limitaciones por ejemplo son que en las pruebas para verificar la funcionalidad del código, tocaba hacerlas directa y manualmente.

- **Check_Satisfiable**

Para la implementacion de esta función me enfoque en la aplicación de model checking por lo que al principio obtuve los átomos presentes en la fórmula y luego generé todos los modelos posibles utilizando la función **get_all_models** previamente implementada. Seguido a esto iteré sobre cada modelo para verificar la fórmula lógica y así determinar si existía al menos un modelo que la hiciera verdadera, retornando dicho modelo cuando se cumplía la condición.

En realidad para esta implementación no tuve ninguna dificultad debido a que dependía directamente de la función previamente implementada y junto con la guía de instrucciones mas la pista que se daba fue muy fácil su realización.

Reflexion de uso de IA

No se utilizó IA para esta función, ya que como se mencionó anteriormente su implementación dependía directamente de la función get_all_models por lo que al reutilizar dicha función, la lógica fue directa y no requirió correcciones externas con apoyo de la IA.

- **Check_Valid**

Para la implementación de esta función me enfoque en revisar la validez de la fórmula evaluandola en todos los modelos, por lo que primero obtuve los átomos presentes en dicha fórmula y después generé todos los modelos utilizado la funcion la misma función previamente implementada **get_all_models.** Al igual que en la función anterior, iteré sobre cada modelo para evaluar la fórmula y verificar si existía alguno donde fuera falsa.

Del mismo modo que la función anterior, para esta implementación no tuve ninguna dificultad debido a que dependía directamente de la función get_all_models y junto con la guía de instrucciones mas la pista que se daba fue muy fácil su implementación.

Reflexion de uso de IA

No se utilizó IA para esta función, ya que como se mencionó anteriormente su implementación dependía directamente de la función get_all_models por lo que al reutilizar dicha función, la lógica fue directa y no requirió correcciones externas con apoyo de la IA.

- **Check_Entailment**

Para la implementación de esta función también me enfoqué en aplicar model checking con tal de revisar si la base de conocimiento implicaba la consulta, por lo que al inicio obtuve todos los átomos presentes tanto en la base de conocimiento como en el query. A partir de esto, generé todos los modelos posibles con la misma función **get_all_models** e hice una iteración sobre cada uno para evaluar los casos donde la base de conocimiento era verdadera y por consiguiente revisar si la consulta tambien se cumplia en esos mismos modelos.

A diferencia de las dos anteriores funciones y aunque tambien se usaba la función get_all_models, en este caso si hubo dificultades pues no toda la implementación dependía en la mayoría de esta función. Una de ellas es que tenía que hacer que el codigo encontrara correctamente los casos donde la base de conocimiento fuera verdadera pero que la consulta fuera falsa. Añadido a esto no lograba entender muy bien el operador lógico **"|="**, por lo que que al no haberlo usado, esto sobreescribía los atomos en cada iteración.

Reflexion de uso de IA

Prompt Utilizado con Chat GPT:

Porfavor colabórame a corregir el siguiente codigo mostrándome los errores que puede haber y además me indicas cual es la correción que realizaste en forma de comentarios. También porfavor optimizala en dado caso que se pueda.

La IA se utilizó con tal de corregir y mejorar el código incial el cual yo habia desarrollado con base en las intrucciones y la pista dada al igual que en las funciones anteriores. En este caso si fueron errores cruciales de estructura y sintaxis de operadores lógicos, que basicamente dañaban toda la implementación en general, por lo que al habermelos recalcado, logré entender mejor la funcionalidad del codigo.

En este caso pienso que las limitaciones y beneficios de la IA en este contexto son los mismos que especifiqué en la primera función implementada: **get_all_models**

- **Truth_table**

Se utilizo un enfoque de model chequing. El algoritmo extrae los símbolos de la formula, genera todas las combinaciones posibles de valores de verdad y evalua la fórmula de cada uno para determinar su valor de verdad resultante. La principal dificultad fue identificar el método correcto para extraer los átomos. Se verifico mediante las pruebas unitarias, asegurando que la tabla generada coincidiera con los valores esperados para conectores básicos.

Reflexion de uso de la IA

En esta función no se usó IA, esto porque no considere que hubiera alguna forma de optimisar la funcion o que hisiera su trabajo de una mejor manera.

REFLEXION PUNTO 2

- **Eliminate_iff**

Se utilizo recursión sobre el arbol. El algoritmo busca nodos de tipo Iff y los reemplaza por la equivalencia lógica. Para los demás nodos, la función se llama a sí misma para asegurar que se eliminen los bicondicionales anidados. La principal dificultad surgió fue entender cómo funcionaba el árbol y sus características para los atoms. La verificación se validó con el test confirmando que tras la ejecución no quedaran instancias de la clase Iff.

Reflexión de uso de la IA

Podrías hacer esta función más optima, corta y que función bien para los test en los que falla

La IA se usó para corregir unas pocas fallas que tenía en los test y además el código original era más largo y complicado de entender a simple vista, algo que la IA corrige en su versión del código.

- **Eliminate_implication**

Similar con la función anterior, se usó un enfoque recursivo. El objetivo fue localizar las Implies y transformarlas siguiendo la regla: $A \rightarrow B \equiv \neg A \vee B$. La mayor dificultad fue saber cómo funcionaba bien el árbol, al igual que la función anterior, a la vez fue entender la parte del código de logic para poder programar correctamente la función. Se usaron los test unitarios.

Reflexión de uso de la IA

Podrías hacer este código más optimo y que se arreglen algunos errores que tiene con los test

Con este prompt la IA modifico la función para que fuera más pequeña y funcionara mejor, a la vez que hiso el código más optimo y corrigiera errores en los test que tenía el anterior código.

- **Push_negation_inward**

Se aplicaron leyes de Morgan mediante un esquema de patrones. La función identifica cuando un operador NOT tiene como hijo un AND o un O, y distribuye la negación hacia los operandos internos, cambiando el operador principal en el proceso. Manejar la naturaleza de los operadores AND y OR fue lo más difícil al momento de hacer la función., se tuvieron que usar comprensión de listas para reconstruir las fórmulas correctamente después de negar cada componente. Se verifico pasando los test unitarios.

Reflexión de uso de la IA

Podrías ayudarme a optimizar este código, que sea más corto y que solucione algunos problemas que tiene con algunos test.

Con este prompt la IA corrigio todos los errores que tenía la versión anterior de la funcion, a la vez que hiso el código más corto, y fácil de entender.

- **distribute_or_over_and:**

En esta función lo que hicimos fue recorrer la fórmula y revisar su tipo. Si es un Atom o un Not, se devuelve igual porque ya no se puede distribuir más en este punto. Si es un And, aplicamos la función recursivamente a cada elemento dentro de formula.conjuncts y reconstruimos el And. El caso importante es cuando es un Or: ahí creamos una lista hijos con los elementos procesados y buscamos si alguno es un And usando and_index. Si no hay ningún And, devolvemos el Or tal como está.

Si sí encontramos un And, lo guardamos en and_child y el resto en others. Luego construimos nuevas_conjunciones, donde por cada elemento c dentro de and_child.conjuncts creamos un nuevo Or(c, *others), aplicando recursión nuevamente. Finalmente devolvemos un And con esos resultados. La dificultad principal fue detectar correctamente el And dentro del Or y manejar bien las listas sin romper la estructura, y se verificó ejecutando los test con python main.py --test hasta que todos pasaran correctamente.

Reflexión de uso de la IA

Podrías ayudarme a optimizar este código, que sea más corto y que solucione algunos problemas que tiene con algunos test.

Con este prompt la IA ayudó a corregir errores de lógica que tenía la versión inicial, especialmente en la parte donde se detecta el And dentro del Or y cómo se reconstruye la fórmula. También permitió organizar mejor las variables y hacer el código más claro y fácil de seguir. La IA fue útil como apoyo para depurar y simplificar, pero igual fue necesario entender cada paso para poder explicar correctamente cómo funciona la distribución.

- **Flatten:**

En esta función lo que hicimos fue eliminar niveles innecesarios de anidación en las fórmulas, especialmente cuando hay And dentro de And o Or dentro de Or. Primero revisamos el tipo de la fórmula: si es un Atom, se devuelve igual porque no tiene estructura interna. Si es un Not, aplicamos flatten recursivamente a su operand. Si es un And, creamos una lista nuevos donde vamos guardando los resultados; para cada elemento c dentro de formula.conjuncts, aplicamos flatten(c) y lo guardamos en c_flat. Si c_flat también es un And, usamos extend

para meter sus elementos directamente en nuevos; si no, usamos append, evitando estructuras como And(And(...)).

Lo mismo se hace para el caso de Or, pero con formula.disjuncts, aplanando Or dentro de Or. Al final revisamos el tamaño de nuevos: si tiene un solo elemento lo devolvemos directamente, y si tiene varios reconstruimos el And o el Or. La dificultad principal fue manejar bien cuándo usar append o extend para no dañar la estructura, y la verificación se hizo ejecutando los test con python main.py --test hasta que todas las fórmulas quedaran correctamente aplanadas.

Reflexión de uso de la IA

Podrías ayudarme a optimizar este código, que sea más corto y que solucione algunos problemas que tiene con algunos test.

Con este prompt la IA ayudó a simplificar la lógica, especialmente en el uso de listas y en cómo manejar los casos donde hay anidación profunda. También ayudó a organizar mejor el código para que fuera más claro cuándo usar append y cuándo usar extend. La IA fue útil para hacer el código más limpio y evitar errores, pero fue necesario revisar cada parte para entender bien cómo se estaba aplanando la estructura.

REFLEXION PUNTO 3

- **veneno_villa_espinas.py**

En este archivo usamos lógica de predicados para representar el caso como relaciones entre personas y objetos. Primero definimos constantes con Term(...) como reynaldo, margot, pablo, bernardo y frasco_arsenico, que representan entidades específicas del problema. Luego agregamos hechos con kb.add_fact(...), como huellas_en(reynaldo, frasco_arsenico) o lejos_escena(pablo). Después construimos reglas con kb.add_rule(...) usando variables como Term("$X") para generalizar, por ejemplo evidencia_directa(X) si alguien tiene huellas en el arma, y culpable(X) si además no tiene coartada verificada. También modelamos descartado, testimonio_confiable, encubridor y coartada_cruzada para cubrir toda la lógica del caso.

La principal dificultad fue decidir qué debía ser un hecho y qué debía ser una regla, además de usar correctamente las variables como $X para que las inferencias funcionaran en todos los casos. También hubo errores iniciales en los nombres de los predicados que hacían fallar los test. La verificación se realizó ejecutando python main.py --test y ajustando la lógica hasta que todas las consultas del caso se resolvieran correctamente.

Reflexión de uso de la IA

Podrías ayudarme a organizar este caso usando lógica de predicados, separando bien hechos y reglas, y asegurando que pase los test.

La IA ayudó a identificar qué cosas debían ser hechos (huellas, coartadas) y cuáles reglas (culpable, encubridor). También ayudó a usar variables como $X correctamente. Fue útil para ordenar la lógica y evitar errores en las inferencias.

- **robo_expreso_sur.py**

En este archivo también usamos lógica de predicados, pero enfocándonos más en testimonios y ubicación. Definimos constantes como elena, victor, don_rodrigo y marquesa, y agregamos hechos con kb.add_fact(...) como en_escena(elena) o grabado_lejos_escena(don_rodrigo, vagon_equipaje). Luego usamos reglas con variables ($X, $Y) para construir inferencias como descartado(X) si alguien estaba lejos de la escena, y testigo_imparcial(X) si es la víctima. A partir de eso, definimos acusacion_creible(X, Y) y finalmente culpable(X) si la persona estaba en la escena y fue acusada de forma creíble. También modelamos defiende_al_culpable y alianza_coartadas para representar cómo los sospechosos se cubren entre sí.

La principal dificultad fue modelar correctamente la credibilidad de los testimonios, especialmente cómo conectar testigo_imparcial con acusacion_creible sin generar conclusiones incorrectas. También fue importante usar bien las variables para que las reglas funcionaran en todos los casos. La verificación se hizo ejecutando python main.py --test y revisando que las inferencias coincidieran con la narrativa del caso y que todos los test pasaran correctamente.

Reflexión de uso de la IA

Podrías ayudarme a modelar este caso con lógica de predicados y asegurar que las reglas representen bien la narrativa.

La IA ayudó a estructurar mejor las reglas, especialmente la parte de acusacion_creible y cómo conectar eso con culpable. También ayudó a entender cuándo usar hechos directos y cuándo usar inferencias.

- **sabotaje_pharmax.py**

En este caso usamos lógica de predicados para combinar varios tipos de evidencia: económica, acceso al lugar y coartadas. Definimos constantes como tec_rios, asistente_mora, dra_santos y director_vega, y agregamos hechos con kb.add_fact(...) como recibio_pagos(tec_rios, syntek_corp), registro_acceso(tec_rios, sala_cultivos) y sin_coartada(tec_rios). Luego usamos reglas con variables ($X, $Y) para inferir conflicto_intereses(X, Y) cuando alguien recibe pagos de una empresa beneficiada, y motivo_economico(X) a partir de eso. También inferimos acceso_en_momento(X) usando

los registros de acceso, y finalmente definimos culpable(X) cuando se cumplen todas esas condiciones juntas. Además, modelamos denuncia_informada para darle más contexto a los testimonios.

La principal dificultad fue combinar correctamente varias condiciones en una sola regla de culpabilidad sin generar errores, especialmente al manejar motivo_economico, acceso_en_momento y sin_coartada. También fue importante usar bien las variables para que las reglas aplicaran correctamente a cada sospechoso. La verificación se realizó ejecutando python main.py --test y ajustando las reglas hasta que solo las personas que cumplían todas las condiciones fueran marcadas como culpables.

Reflexión de uso de la IA

Podrías ayudarme a combinar correctamente motivo, acceso y coartada en reglas de lógica de predicados.

La IA ayudó a estructurar la regla de culpabilidad con varias condiciones al mismo tiempo. También ayudó a identificar cómo convertir hechos financieros en motivo_economico. Fue útil para evitar errores en reglas complejas.

- **herencia_hacienda_rosal.py**

En este archivo usamos lógica de predicados para modelar el caso a partir de la herencia y la evidencia física. Definimos constantes como sobrino_esteban, abogado_restrepo, secretaria_luna y enfermera_campos, y agregamos hechos con kb.add_fact(...) como huellas_en(sobrino_esteban, vaso_adulterado), hereda_actual(X) y pierde_con_cambio(X). A partir de esto, usamos reglas con variables ($X) para inferir motivo_doble(X) cuando alguien hereda actualmente pero perdería con el cambio, y evidencia_fisica(X) cuando tiene huellas en el objeto del crimen. Luego definimos culpable(X) combinando motivo, falta de coartada y evidencia, y también modelamos desvio_sospechoso, encubridor y acusacion_corroborada para analizar cómo se comportan los sospechosos.

La principal dificultad fue modelar correctamente el motivo_doble, ya que depende de dos condiciones al mismo tiempo, y asegurarse de que se conectara bien con la evidencia física en la regla de culpabilidad. También fue importante manejar bien las variables para que las reglas funcionaran correctamente en todos los casos. La verificación se realizó ejecutando python main.py --test y ajustando las reglas hasta que el sistema identificara correctamente al culpable según la lógica del caso.

Reflexión de uso de la IA

Podrías ayudarme a representar correctamente el motivo doble y la relación con la evidencia física.

La IA ayudó a separar bien las condiciones del motivo (hereda_actual y pierde_con_cambio) y a conectarlas con la culpabilidad. También ayudó a estructurar mejor las reglas de acusación.

- **red_puerto_sombras.py**

En este caso usamos lógica de predicados para modelar una red criminal donde hay más de un culpable. Definimos constantes como oficial_duarte, marinero_pinto, capitan_herrera e inspector_nova, y agregamos hechos con kb.add_fact(...) como firma_manifiestos_fraudulentos(oficial_duarte), acceso_bodega(marinero_pinto) y sin_coartada(...). A partir de esto, usamos reglas con variables ($X, $Y) para inferir fraude_documental(X) e introduce_contrabando(X), y luego definimos culpable(X) dependiendo de esas acciones junto con la falta de coartada. También modelamos relaciones como comparten_red(X, Y) y operacion_conjunta(X, Y) para conectar a varios sospechosos, además de red_activa(R) para identificar si la red sigue operando.

La principal dificultad fue modelar correctamente las relaciones entre múltiples culpables sin generar inferencias incorrectas, especialmente en comparten_red y operacion_conjunta. También fue importante asegurar que la culpabilidad individual estuviera bien definida antes de conectar a los sospechosos como red. La verificación se realizó ejecutando python main.py --test y revisando que el sistema detectara correctamente tanto la culpabilidad individual como las relaciones dentro de la red.

Reflexión de uso de la IA

Podrías ayudarme a modelar relaciones entre múltiples culpables dentro de una red usando lógica de predicados.

La IA ayudó a estructurar las reglas que conectan a varias personas, especialmente comparten_red y operacion_conjunta. También ayudó a entender cómo representar una red activa a partir de individuos culpables.

RESPUESTA PREGUNTAS PUNTO 3.b

---

1. **¿Cómo se modela la diferencia entre evidencia directa (por ejemplo, huellas en el arma) e indirecta (por ejemplo, un testimonio) usando predicados y reglas?**

En el código diferenciamos esto usando predicados distintos y reglas diferentes. La evidencia directa la modelamos con cosas físicas como huellas_en(X, O) y arma_crimen(O). Luego usamos una regla que junta esas dos cosas para crear algo más fuerte: evidencia_directa(X). Es decir, primero definimos hechos concretos con kb.add_fact(...) y luego una regla tipo: si una persona tiene huellas en el objeto que es el arma del crimen, entonces tiene evidencia

directa. En cambio, la evidencia indirecta la modelamos con predicados como acusa(X, Y) o da_coartada(X, Y). Esos no dicen nada por sí solos, entonces creamos reglas adicionales como testimonio_confiable(X, Y) que dependen de otras condiciones, por ejemplo que el que acusa esté descartado(X). Entonces la diferencia en el código es clara: la evidencia directa sale de objetos físicos y reglas simples, mientras que la indirecta necesita más pasos y validaciones.

2. **¿Qué sucede si un sospechoso tiene tanto evidencia en su contra como coartada verificada? ¿El motor detecta esta contradicción? Explica con un ejemplo.**

En nuestro código, la evidencia y la coartada generan conclusiones diferentes. Por un lado, la evidencia puede llevar a culpable(X) con reglas como: si hay evidencia_directa(X) y además sin_coartada_verificada(X), entonces es culpable. Por otro lado, la coartada verificada se modela con predicados como coartada_verificada(X) que luego llevan a descartado(X). El problema es que nunca definimos una regla que diga que alguien no puede ser culpable y descartado al mismo tiempo. Entonces, si en un caso hipotético una persona tuviera ambos predicados, el motor no marca error ni contradicción automáticamente, simplemente puede aceptar ambos resultados. Por ejemplo, si alguien tuviera huellas_en el arma (evidencia directa) pero también doc_ausencia_pais (coartada verificada), el sistema podría derivar dos cosas distintas si las reglas lo permiten. Esto pasa porque el motor no maneja contradicciones a menos que uno las modele explícitamente.

3. **¿En qué se diferencia razonar con forward chaining vs. backward chaining al resolver un caso? ¿Cuándo conviene usar cada uno?**

En el código usamos ambos enfoques y se diferencian en cómo recorren las reglas. El forward chaining parte de los hechos que agregamos con kb.add_fact y empieza a aplicar todas las reglas posibles para generar nuevos predicados como culpable, descartado, etc. Es como dejar que el sistema descubra todo automáticamente. En cambio, el backward chaining empieza desde una pregunta específica, por ejemplo culpable(reynaldo), y trata de ver qué reglas podrían probar eso, revisando si los predicados necesarios existen. Entonces, forward chaining sirve cuando queremos obtener todas las conclusiones posibles del caso (por ejemplo, todos los culpables), mientras que backward chaining es mejor cuando solo queremos responder una consulta puntual sin calcular todo lo demás.

4. **¿Qué rol juegan las variables vs. las constantes en la expresividad de las reglas? ¿Qué se perdería si solo se usaran constantes?**

En el código usamos constantes como Term("reynaldo") o Term("margot") para representar personas específicas del caso. En cambio, usamos variables como Term("$X") y Term("$Y") dentro de las reglas para hacerlas generales. Por ejemplo, una regla como culpable(X) no está escrita para una persona específica, sino que puede aplicarse a cualquiera que cumpla las condiciones. Esto es clave porque permite reutilizar las reglas en todos los casos. Si solo usáramos constantes, tendríamos que escribir una regla diferente para cada persona, lo cual

haría el sistema muy repetitivo y poco escalable. Básicamente, sin variables el sistema perdería la capacidad de generalizar y se volvería muy limitado.

5. **¿Cómo afecta la suposición de mundo cerrado (lo que no se puede probar es falso) a las conclusiones sobre sospechosos que no tienen evidencia en su contra ni coartada verificada?**

En nuestro sistema, si algo no está en la base de conocimiento, simplemente no se puede inferir. Por ejemplo, si no existe ninguna regla o hecho que lleve a culpable(X), entonces el sistema no considera a esa persona culpable. Lo mismo pasa con descartado(X). Esto es lo que se conoce como mundo cerrado: lo que no se puede probar, se asume falso. En la práctica, esto hace que un sospechoso sin evidencia ni coartada simplemente "no aparezca" en las conclusiones. El sistema no lo marca como culpable ni como inocente, pero implícitamente lo trata como si no fuera culpable porque no se puede demostrar. Esto simplifica el modelo, pero no refleja completamente la realidad, porque en la vida real la falta de evidencia no significa inocencia.